

SELF-INTRODUCTION

My self **D.Sai Pradeep** . I started my career as a **TEST EXECUTIVE 4 years** back with **DEEGEE Software PVT LTD**. Currently I have been working as a **Software Test Engineer**.

My responsibility is to understand Business Requirement Specification (BRS) and high-level scenarios. And converting them into manual test cases and automation scripts. Execution of test cases and reporting of defects to the developer. I have experience on functional tests like manual and automation tests like Regression, Smoke, Sanity.

My Project is being tested on automation where we have been using Selenium with Java and TestNG framework. And we have been using Page Object Model as well. For build management tool we have been using Maven, for version control we have been using Git and for defect management tool we are using Jira.

My Project is worked in Agile environment We have sprint cycle for 2 weeks ya that's it.

DAY TO DAY ACTIVITIES

First thing I do after login in my system. I check the active sprint in Jira for our project code. There I can see my assigned open tasks. After that I will check my mail if there is any important mail, I need to take action on it. Then we have our daily scrum meeting where we exactly discuss about what did we yesterday and what will we do today. If we have any blockers, Product owner and scrum master help us to resolve those blockers. After that I need to take the pending task and do needed action whether creating test case, Execution, Defect retesting if any.

TestNG FRAME WORK STRUCTURE

- Currently we are using TestNG framework with page object model.
- For build the framework we have used maven
- In our framework mainly consists of 2 packages like src/main java and
Src / test java in src /main java we have create a utility package in that utility package contains all the usable classes required for the project like generic methods, xls reader, read data ...etc,
- In src/test package we created two separate packages for pages and tests
- In page package used to maintain a separate class for every webpage and the test package is used to run the code of those page class after all that we create a different types of folders
- Like logs ,object repo properties , test output , test data , test reports , screenshots , pom.xml
- Logs : logs are used to store the test case activities while running
- Object repo : in that we contain a config. properties file which is used to give browser name and url and required details for the project.

- Test output : in that folder we contain a TestNG-failed .xml file this file is used to save and run only failed test cases.
- Test data : in that folder we maintain the test data in excel format required for the project by using that folder we get the data
- Test report folder is used to store the reports generated from the test cases
- Screenshots folder is used to store the screenshots for the test cases
- In maven we have pom.xml file and that pom.xml file is used to add dependencies, plugins required for the project.
- This pom.xml makes us to update the dependencies by changing the version data in the pom.xml file without opening maven repository
- Groups.xml : to separate the test cases like sanity , smoke ,regression etc. like that

CUCUMBER FRAME WORK STRUCTURE

- Currently we are using Cucumber framework with page object model.
- For build the framework we have used maven
- Cucumber Framework is a popular tool used for Behaviour-Driven Development (BDD).
- Cucumber supports BDD, which emphasizes collaboration between business stakeholders and developers.
- Test scenarios are written in Gherkin language, a simple syntax that uses a Given-When-ThenAnd structure.
- In our framework mainly consists of 2 packages like src/main java and Src / test java in src /main java we have create a utility package in that utility package contains all the usable classes required for the project like generic methods, xls reader, read data ...etc,
- In mainly cucumber have some components are available.
 - 1.Featurefile: Contains Gherkin scenarios (. feature files). Located in the src/test/resources folder.
 - Feature file contain scenario and scenario Outline, Background and example keywords.
 - Scenario – To Executes a single set of steps.
 - Scenario Outline – It Supports data-driven testing by running steps multiple times with different data sets.
 - Background – It is used to write the all-common steps for all scenarios within a feature file. • Example – it consists of list of steps. Example key word to provide the data for template.
 - 2.Stepdefination: it is used for every keyword corresponding method will be implemented. • Each and every steps are implemented under Step definition file.

Cucumber Step Definition (Package):

- a. Where we will write methods and logics.
- b. We can generate these methods by running Runner class.

c. Here, just we need to add our logics as per methods.

Note: We can mention tags in. feature files.

Ex: @Sanity Testing. @Regression Testing.

We use these types of tags for Grouping.

- 3.Runnerpackage: to execute scenarios (or) Executes the feature files using a test runner like JUnit or TestNG.

Cucumber Runner (Package):

- a. Test Runner (Class) ⑦ @Run with (cucumber. Class), @Cucumber.options(features = “path”, glue = “step definition package name”)
 - b. Groups Runner (Class) ⑦ (@Run with (cucumber. Class), @Cucumber.options(features = “path”, glue = “step definition package name”, tags = {“@Testing name”})) ○
- Parameterization: if we use parameter in cucumber, we use data tables and scenario outline.

- 4.Hooks: Hooks is a common snippet of code, which can be executed globally before and after scenario. that can be executed before or after specific test scenarios or steps.

Cucumber provides several hooks that can be used.

- @Before - Execute before each scenario. Example: Launching a browser, opening a database connection.
 - @After - Executes after each scenario. Example: Closing a browser, deleting temporary test data.
 - @BeforeStep - Executes before each step within a scenario. Adding logs or printing step details for debugging.
 - @AfterStep - Executes after each step within a scenario. Commonly used for capturing screenshots or logging.
- Advantages : Cucumber is a readable and understandable ,reusability , collaboration ,and it is a open source.
 - after all that we create a different types of folders
 - Like logs, object repo properties, test output, test data , test reports , screenshots , pom.xml, Groups.xml
 - Logs: logs are used to store the test case activities while running
 - Object repo: in that we contain a config. properties file which is used to give browser name and URL and required details for the project.
 - Test output: in that folder we contain a TestNG-failed .xml file this file is used to save and run only failed test cases.
 - Test data: in that folder we maintain the test data in excel format required for the project by using that folder we get the data
 - Test report folder is used to store the reports generated from the test cases
 - Screenshots folder is used to store the screenshots for the test cases

- In maven we have pom.xml file and that pom.xml file is used to add dependencies, plugins required for the project.
- This pom.xml makes us to update the dependencies by changing the version data in the pom.xml file without opening maven repository
- Groups.xml: to separate the test cases like sanity, smoke, regression etc. like that

Project#1

Project Name : MetLife

Duration : September 2020 - November 2021 Project

Description:

MetLife, Inc. is a leading global provider of insurance, annuities and employee benefit programs. Through its subsidiaries and affiliates, MetLife holds leading market positions in the United States, Japan, Latin America, Asia, Europe, the Middle East and Africa. Performed Automation Testing for MetLife applications. The testing team consists of functional manual testing team and automation testing (Selenium). Performing Automation Testing for MetLife applications.

PROJECT#2

Project Name : Shirt App

• Role : Automation Tester

• Duration :December 2021 – Till Date Project

Description:

Shirt App is the ultimate solution for shirt lovers, offering a cutting-edge shopping experience tailored specifically for shirt enthusiasts. With a user-friendly interface, a vast catalog of shirt styles, and innovative features, Shirt App brings the world of fashion to your fingertips, making it easier than ever to discover, purchase, and style your perfect shirts.